DroneDream

AURORA: Agentic UAV Refinement through Optimization, Reflection, and Assurance

Chi Zhang

Abstract

UAV controller tuning in simulation must bind every Candidate to an intended scenario, finite budget, objective and constraint policy, trustworthy execution, and holdout boundary. Foundation models can interpret intent and coordinate methods, but fluent output cannot prove execution. **AURORA—Agentic UAV Refinement through Optimization, Reflection, and Assurance**—is the evidence-gated Harness inside DroneDream 1.0. It compiles bounded context, filters tool eligibility before routing, selects versioned optimizers or deterministic fallbacks, executes immutable Trial, Candidate, and Job snapshots, verifies claim receipt v3 and Outcome Contract 2.20, isolates fault and holdout evidence, and freezes selection only after deterministic checks pass. Models may propose and route work; software contracts retain authority over parameter domains, state transitions, evidence admission, and completion. Evidence 2.7 compiles bounded observational reflection from verified decision cohorts. It admits learning only when the execution receipt and every modern Candidate receipt agree; incomplete, legacy, or drifted cohorts make the block unavailable, and no association becomes causal tool credit. Exact-commit evidence records 1,147 backend tests in 788.78 seconds plus an 81-check focused supplement on the same software subject; Windows Rust was not run. Frozen software artifacts further record 16/16 deterministic policy-holdout checks, 25/25 guarded source-contract probes versus 6/25 weakened, and 10/10 matched fallback comparisons over 15 Jobs and 579 Trials. A network-free four-arm ablation spans 20 Jobs and 554 Trials, while its receipt-only-memory isolation remains inconclusive in 5/5 seed blocks. The current Evidence 2.7 / Prompt 1.6 provider freeze grades 24/24; Prompt 1.5 histories grade 19/24 (unqualified) and 21/24 (qualified), exposing unpaired stochastic variation rather than causal prompt lift. These results validate exercised routing, provenance, reflection, and recovery contracts, not general optimizer superiority, physical simulator quality, real-aircraft safety, or simulation-to-reality transfer. Customer qualification still requires matched multi-seed PX4/Gazebo outcomes under frozen conditions, with retained failures and independent recomputation before any public deployment, release decision, or customer-facing claim about simulator or optimizer performance.

github.com/ChiZhang-805/DroneDream

1. Introduction

UAV controller tuning is not a single-number search. PX4 layers rate, attitude, velocity, and position controllers, and useful gains shift with propulsion, mass, assembly, and scenario conditions. Its official workflow therefore remains iterative: apply parameters, excite the system, inspect the response, and revise. Software-in-the-loop changes the risk and cost of that loop, not its combinatorics. A credible campaign must choose a method and finite budget, preserve scenario identity, evaluate objectives and constraints, distinguish parameter failure from infrastructure fault, and reserve unseen scenarios for final selection. Derivative-free and Bayesian methods systematize search, from reusable services to trust-region and sparse high-dimensional variants [*2*, *3*, *4*, *5*, *6*, *7*, *8*], but still assume a correctly formed experiment. They neither translate engineering intent into an auditable program nor prove that its declared conditions were executed as declared in its persisted manifest.

Foundation models provide a useful interface layer, but not experimental authority. Tool-using agent systems show that model reasoning becomes more effective when coupled to explicit tools, machine-readable state, feedback, reflective memory, and bounded execution interfaces [*16*, *17*, *28*, *29*, *30*]. In control optimization, however, a plausible explanation cannot prove that a scenario was applied, a metric came from the intended trial, or a final candidate remained isolated from holdout leakage. Those facts must come from deterministic software and retained evidence. AURORA (Agentic UAV Refinement through Optimization, Reflection, and Assurance), the evidence-gated Harness inside DroneDream 1.0, therefore uses model intelligence to interpret the task and rank specialized methods, while deterministic compilers, versioned contracts, simulator receipts, outcome taxonomy, and a final freeze gate authorize every consequential transition. The model may propose the next action; it cannot declare that action executed or promote an unverified observation. AURORA therefore compiles conversation into a bounded optimization program whose claims are earned by retained evidence.

This report makes three contributions. First, it specifies the system boundary that separates model reasoning, tool eligibility, simulation execution, and evidence authority. Second, it reports reproducible routing, guard, fallback, and synthetic integration evidence without relabeling software contracts as physical performance. Third, it defines the experiments still required before broader simulator-quality claims are justified. The presentation follows the causal narrative common to modern technical reports in agentic and embodied AI [*15*, *18*, *19*], while keeping DroneDream's claims tied to its own frozen artifacts and independently auditable release boundary for independent regeneration or reuse.

1.1 Product scope

The product boundary is simulation-first. The desktop application guides scenario design, experiment planning, execution, evidence review, recovery, and reporting. The public website explains and distributes the product, hosts documentation and community features, and exposes subscription entry points. Supabase supplies identity, community data, quota accounting, and the managed-model gateway; DroneDream 1.0 keeps PX4/Gazebo simulation local to the owned Runtime. The boundary is equally explicit about what is not claimed: this report does not certify real-aircraft safety, guarantee simulation-to-reality transfer, or authorize unbounded autonomous flight actions. A model may propose or route work, but it may not convert an unverified result into a completed tuning outcome. AURORA must earn that transition with evidence.

2. Related work and design rationale

Black-box optimization supplies the search machinery, but not the experimental constitution. CMA-ES provides a robust derivative-free evolutionary strategy, while Vizier and BoTorch formalize reusable optimization services and modern Bayesian-optimization primitives. TuRBO addresses local search in high dimensions, and SAASBO models sparse axis-aligned structure [*4*, *5*, *6*, *7*, *8*]. Efficient Global Optimization established the surrogate-and-acquisition loop for expensive black-box functions, practical Bayesian optimization showed how to carry that loop into noisy model selection, and the field review makes the remaining choices around priors, acquisition, constraints, and parallelism explicit [*31*, *32*, *33*]. These methods motivate DroneDream's portfolio, yet each remains conditional on a correctly specified objective, feasible domain, trial budget, and trustworthy observation stream. DroneDream's distinct engineering problem begins one level above the optimizer: it must decide which method is eligible, prove which scenario was actually applied, and prevent infrastructure faults or holdout outcomes from entering the learning history or contaminating any later comparison.

Robotic foundation models demonstrate broad semantic and action priors, but they solve a different authority problem. RT-1 and RT-2 established large-scale vision-language-action learning; Open X-Embodiment and OpenVLA expanded cross-embodiment data and open model access; π_0 and $\pi_{0.5}$ explored flow-based action generation and heterogeneous co-training; JoyAI-RA continued this line with a multi-source autonomy foundation model [*9*, *10*, *11*, *12*, *13*, *14*, *15*, *20*, *21*, *27*]. DroneDream does not claim an end-to-end flight policy or place a language model in the control loop. It uses model reasoning only to interpret intent and rank eligible tools; deterministic software retains authority over experiment construction, simulator execution, evidence admission, final selection, release, and rollback.

Data scale and benchmark breadth have advanced faster than evidence semantics. Open X-Embodiment, BridgeData V2, DROID, and AgiBot World expand the diversity of robot experience; RoboCasa and RoboTwin 2.0 scale simulation tasks and controlled perturbations [*11*, *22*, *23*, *24*, *25*, *26*]. These resources strengthen policy training and comparative evaluation, but their success metrics do not answer whether a local UAV tuning run applied the intended scenario, whether a failed trial was physical or infrastructural, or whether holdout evidence leaked back into search. DroneDream therefore treats dataset coverage, optimizer quality, and evidence validity as independent axes: benchmark breadth or policy scores cannot substitute for evidence about the exact experiment that generated a reported result.

Tool-using agents clarify why that separation matters. SWE-agent shows the value of an explicit agent-computer interface, and AlphaEvolve couples model proposals to evaluators and an evolutionary loop [*16*, *17*, *28*, *29*, *30*]. ReAct interleaves reasoning with environmental actions; Toolformer studies learned tool invocation; Reflexion uses feedback as episodic memory. Together they locate capability in interfaces, evaluators, and feedback loops. DroneDream adopts that interface principle but never treats free-form reflection as experimental evidence. Only structured, provenance-bound, policy-eligible outcomes may update decision memory; failure taxonomy, holdout isolation, and the final freeze gate prevent generated narratives from replacing the versioned simulator receipts and outcome records that support a claim.

3. System and method

The architecture assigns a single authority to every transition. The desktop owns user intent and interaction; the FastAPI backend and worker validate job transitions and persist bounded state; the AURORA compiles context, filters tool eligibility, and admits only trusted outcomes; the owned Runtime supplies pinned PX4/Gazebo dependencies; and cloud services provide identity, community data, quota accounting, and a managed-model gateway without exposing the platform key to clients. Separating these roles prevents a router choice from becoming an executable experiment until deterministic compilers, validators, and Runtime probes agree on the same versioned contract before launch or retry.

3.1 AURORA: the evidence-gated Harness

AURORA is a constrained decision system around simulation. Context binds parameters, scenarios, history, budgets, and failures. Capability eligibility removes incompatible tools, while a phase gate exposes only methods suited to exploration, recovery, refinement, diversification, or verification. The router ranks that executable surface and the optimizer proposes Candidates. Verification forces every reduced-fidelity strategy to fidelity 1.0 before acceptance. Provenance-bound memory admits trusted outcomes, while the final gate freezes holdout results and rejects incomplete artifacts. Each stage narrows authority, so fluent output cannot bypass an executable contract or test. The retained decision trace records the selected tool together with every filtered alternative and its rejection basis during later replay.

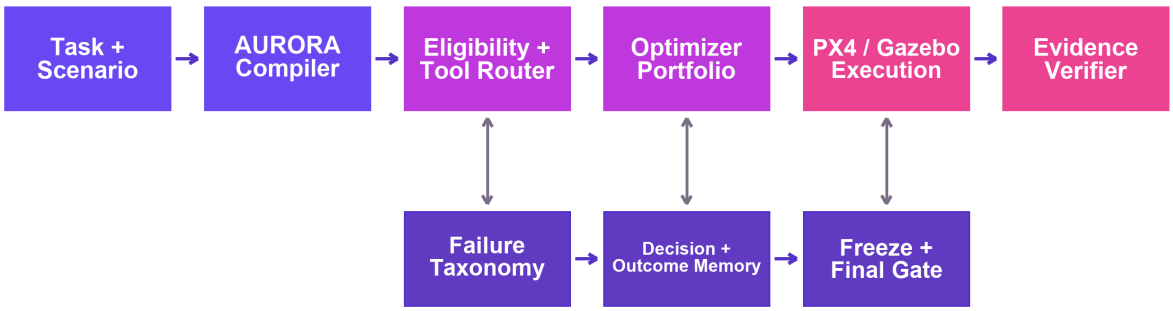

Figure 1. AURORA's evidence-gated optimization loop inside DroneDream 1.0. Provenance-complete, policy-eligible outcomes alone may influence routing memory or pass the final selection gate.

- Desktop boundary — readiness, user input, progress, settings, and safe exit are authorized only by terminal UI state; entry remains hidden while any required launch check is pending or failed, and active work or unsaved drafts trigger a recoverable, attributable exit path rather than silent process termination under the same readiness record for audit use.
- Backend boundary — drafts, Jobs, orchestration, APIs, and artifacts cross the service boundary only through validated state transitions and stable failure codes; rejected transitions retain explicit reasons, source-bound identifiers, and replayable receipts instead of becoming persisted, dispatched, or published work before independent replay.
- AURORA authority — context, eligibility, routing, reflection, and evidence memory admit only provenance-complete, policy-eligible evidence; holdout, infrastructure-fault, legacy, incomplete, or drifted records remain unavailable to routing, final selection, and every later replay that could otherwise reinterpret them as admissible learning evidence.
- Runtime boundary — PX4/Gazebo and pinned dependencies require effect and source evidence from the launched process, accepted customer Runtime, and published release ledger, so each physical claim must bind the executable effect, expanded source, dependency commits, and an independently checked receipt before release and public use.
- Cloud boundary — authentication, community data, quota ledger, and model gateway remain server-authoritative; provider secrets never enter desktop logs or artifacts, and each managed-model debit must retain an authenticated account, quota-ledger entry, provider disposition, and reconcilable service receipt under its frozen billing contract.

Claim-time inputs are immutable before launch. Schema **dronedream.trial-execution-attempt-claim/v3** and digest **execution_input_sha256** bind Trial, Candidate, and Job inputs in an immutable snapshot. Outcome Contract 2.20 authorizes the scenario matrix and constructs **TrialContext** without rereading mutable rows. Checks classify changed inputs as **INPUT_EVIDENCE_DRIFT**, coordinated rewrites as **OUTCOME_CONTRACT_DRIFT**, and unauthorized fields as **SCENARIO_CONTRACT_DRIFT**. Completion locks Job before Trial; historical receipts remain verifiable, new claims use v3, and the original projection stays available for audit replay under the same claim boundary.

Physical claims are concurrency-safe and fairly scheduled. Execution Claim Gate 1.1 combines PostgreSQL FOR UPDATE SKIP LOCKED, a conditional lease update, bounded retries, and an attempt-count fence. Concurrent workers converge on a unique physical execution and accepted outcome for each logical Trial, with no duplicate evidence across the exercised pool. Scheduling Gate 1.0 chooses the least-served eligible Job and preserves FIFO within it. A paired-Job regression alternates sequential claims, preventing desktop-lane monopoly; hosted priority and resource-cost scheduling remain outside the current local scheduling contract, audit replay, and evidence envelope for hosted work.

Model feedback is case-separated. Prompt Schema 2.3 assigns provider-safe training_case_N aliases in frozen order, keeping same-type cases with different weights or numeric configurations distinct. Each admitted case carries its weight, seed count, metrics, and closed failure counts; raw IDs, unsupported text, and holdout evidence stay outside the prompt. This removes feedback-grouping ambiguity without expanding model authority or claiming simulator fidelity, and preserves canonical order when matched Trials arrive in reverse. The separation is preserved in every provider request and retained routing receipt for later independent inspection and audit replay across reruns and reviews over time.

Reproducibility extends below the random seed. A non-unique eigenvector factor can rotate an identical seeded normal sample when another BLAS kernel resolves repeated covariance eigenvalues differently. DroneDream instead uses the unique symmetric positive-definite covariance square root and canonical unit-vector spelling before discrete projection. The frozen optimizer transcript hashes executed candidates, losses, feasibility, route, fidelity, and cohort position, while non-causal GP/CMA diagnostic matrices remain outside that decision hash [*1*]. This boundary lets supported machines replay identical candidate sequences across machines before any provider decision receives credit. The transcript is compared at the executed-Candidate boundary rather than against optimizer diagnostics that may vary by library implementation. A replay mismatch therefore identifies a decision-relevant divergence before a provider route or reported observation is accepted, keeping numerical implementation drift separate from evidence drift during independent replay and release review.

Feedback is verified again at the model boundary. Optimizers, the closed-tool router, and direct proposer share a training-feedback compiler. It reconstructs canonical Trial rows and checks the content-addressed outcome against Candidate identity, generation, parameters, and Trial evidence before exposing a score. Sibling fields cannot override those values. Divergence is quarantined rather than turned into a bad-parameter penalty; migration-era rows remain explicitly unsealed and ineligible for learning, routing memory, or final selection [*1*]. The rule preserves auditability across both current and migration-era experiment histories and their later replay without retroactive reinterpretation across release boundaries.

Reflection is compiled from verified outcomes rather than free-form model self-report. Evidence 2.7 joins the accepted execution receipt to the exact optimizer Candidate cohort for that generation, then requires every Candidate to carry a current v2 evidence receipt and verified training feedback. Prompt 1.6 and Decision Trace 1.3 also bind the exact phase-qualified tool surface. The bounded **observed_outcome** block records cohort size, accepted attempts, optimizer-learning Trials, domain failures, feasible Candidates, completion rate, the incumbent before and after the cohort, and observed absolute and relative improvement. A count mismatch, incomplete result, legacy record, or evidence drift makes the whole block **unavailable**; a zero-dispatch decision is explicitly not applicable. Holdout measurements, private identifiers, seeds, hashes, and failure prose remain outside provider-visible memory. The prompt treats these values only as bounded temporal associations: reflection may guide the next decision, but it cannot assign causal reward or portfolio-child credit unless a separately controlled intervention supports that claim.

- COMPLETED is admitted as successful optimizer evidence only when a usable metric and modern receipt bind the exact Candidate, Trial, scenario, objective direction, and execution provenance; a terminal label without that complete chain remains unavailable for optimizer learning, final selection, export, or any later performance statement in release.
- FAILED with a trusted domain failure code is admitted as a non-success because it may identify an infeasible or unsafe simulated parameter region without being mislabeled as infrastructure failure; the failed attempt stays in the optimization denominator with its scenario and Candidate identity intact through every later aggregate and arm comparison.
- Infrastructure faults, cancellation, malformed evidence, and conflicting terminal states are quarantined and cannot update search or routing memory under any later decision path or replay; each rejected row retains a stable quarantine reason and remains visible to operational diagnostics and independent evidence accounting without later relabeling.
- Holdout-only outcomes remain sealed for final evaluation, never return to training history or provider-visible reflection, and become selectable only through the frozen final gate under the predeclared metric and admissibility policy.

Infrastructure failures, cancellation, malformed or internally conflicting evidence, and holdout-only outcomes are isolated from optimizer learning and routing memory. Admission requires a terminal state, an allowed domain failure code where applicable, a content-addressed Candidate-to-Trial match, and complete provenance for the executed scenario. Rejected rows retain a stable quarantine reason for diagnosis but cannot update scores, route preferences, recovery memory, or the next candidate proposal. This evidence boundary prevents a simulator crash, stale callback, or evaluation-only result from being learned as if it described the quality of a parameter set. The retained experiment ledger makes every quarantine decision independently inspectable without allowing the rejected observation to affect later search.

4. Experiments and evidence

This section keeps unlike evidence units separate. Development and policy-holdout corpora test tool eligibility; source ablations test named guards; the fallback campaign tests recovery equivalence under a fixed mock budget; and synthetic integration tests orchestration on a declared non-physical objective. Every result is recomputed from persisted records and paired with a claim boundary. PX4/Gazebo assertions remain isolated until the signed customer Runtime passes a provenance-bound acceptance campaign, preventing software checks and proxy outcomes from being pooled into one headline score or silently promoted beyond their declared evidence class in any publication-facing claim.

4.1 Development routing evidence

The current provider artifact spans eight routing categories and binds its corpus, Evidence 2.7 inputs, Prompt 1.6, model snapshot, JSON schema, and scoring rule. Predeclared acceptable sets let the report independently regrade every persisted response. The first Prompt 1.5 freeze scored 19/24 and was unqualified by its local-progress threshold; a second freeze under the same Prompt 1.5 template scored 21/24 and qualified, preserving provider stochasticity. Prompt 1.6 states the lower-is-better loss direction, and its current freeze grades 24/24 acceptable selections across all categories. This is development-routing qualification, not a causal prompt-lift estimate: the three unpaired calls do not identify a causal prompt effect, lower simulator loss, or generalization across unseen tasks, vehicles, scenarios, or provider outputs.

Table 1: Current and historical routing accuracy with claim boundary

Comparator	Acceptable selections	Evidence treatment
Prompt 1.6 current freeze	24/24	Current development qualification
Prompt 1.5 first freeze	19/24	Unqualified
Prompt 1.5 repeat freeze	21/24	Qualified historical run
Shared Prompt 1.5 misses	3 cases	Local progress, mixed history, and tight budget
Best constant policy	14/24	Deterministic corpus baseline
Uniform random expectation	5.625/24	Analytical expectation

The failure sets expose where qualification changed. The first Prompt 1.5 freeze missed five cases: two local-progress choices, one sparse-history choice, one mixed-history choice, and one tight-budget choice. The 21/24 repeat retained the local-progress, mixed-history, and tight-budget errors, so three misses persisted across both calls. Prompt 1.6 selected acceptable tools for all 24 current cases after making the loss direction explicit. Because model calls, ordering, and random effects were not paired, this sequence localizes observed routing weaknesses but cannot attribute their removal to the prompt revision. A controlled paired study is therefore required before any wording change receives causal credit.

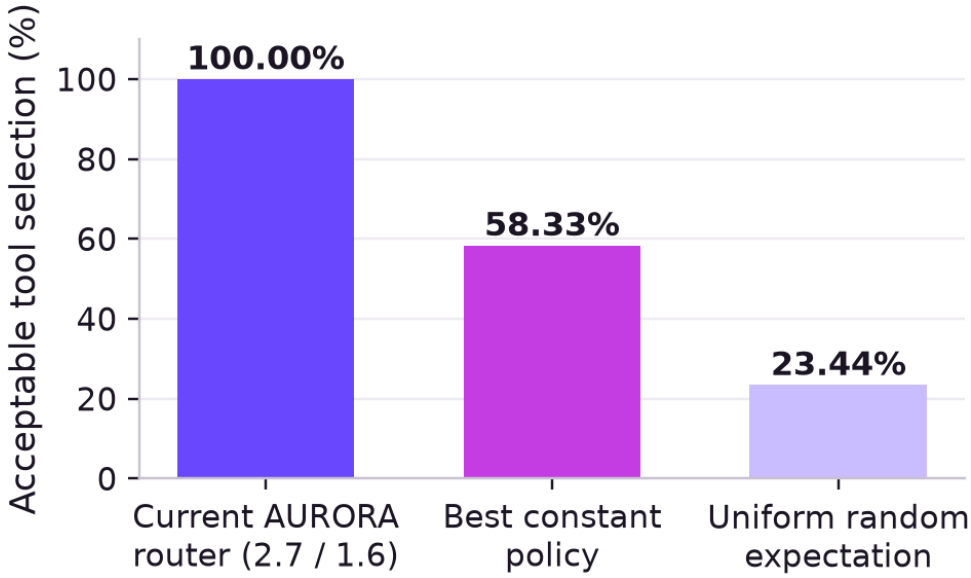

Figure 2. Current acceptable tool selection under Evidence 2.7 / Prompt 1.6. The 24-case result qualifies this frozen development-routing contract but is not a causal prompt comparison or simulator outcome.

4.2 Deterministic routing-policy holdout

A separate 16-case corpus freezes exact eligible-tool sets at capability boundaries and binds the corpus, case IDs, compiled policy inputs, and development-corpus hash. It passes 16/16 exact comparisons. The runner performs zero model calls, simulator runs, or feedback writebacks, and a central flow guard rejects use of these results as prompt examples, training data, development evidence, or runtime feedback. Cases exercise dimension, constraint, multi-objective, fidelity, feasible-history, and restart boundaries so the exact-set assertion detects both an incorrectly admitted tool and an incorrectly excluded one. Because labels become visible when the repository is published, this artifact verifies deterministic policy behavior but is not described as a permanently blind benchmark.

Table 2: Deterministic routing-policy holdout

Measure	Result	Claim boundary
Eligible-tool sets	16/16 exact matches	Current production eligibility only
External activity	0 calls; 0 simulator runs	No model or outcome evidence
Adaptive writeback	0; flow guard rejects	Evaluation artifact only
Label visibility	Published with repository	Not permanently blind

4.3 AURORA source-contract ablations

The ablation suite asks a narrow question: whether each named source guard changes an otherwise identical deterministic decision at the boundary it is designed to protect. For every guard, the production contract and one intentionally weakened comparator receive the same fixed probes, expected result, and scoring rule. Production satisfies 25/25 probes, whereas the matched weakened variants satisfy 6/25 after their designated protection is removed. This construction demonstrates enforceable behavior for fallback, provider trust, outcome isolation, and tool eligibility; it is not a statistical effect estimate and does not measure optimizer quality, simulated control performance, flight safety, or readiness. The comparison is diagnostic because each weakened arm changes one named admission rule while leaving inputs and expected outputs fixed. A failed probe therefore localizes enforcement coverage at the source boundary; it does not estimate how frequently that boundary is reached in customer workloads or how much any optimizer improves a controller in release decisions.

Table 3: AURORA source-contract ablations

Guard	Protected boundary	Cases	Production	Weakened
deterministic fallback	Provider-independent recovery path	4	4/4	1/4
provider trust filter	Trusted provider evidence enters decisions	5	5/5	2/5
scenario/outcome isolation	Holdout and fault evidence stay outside learning	6	6/6	2/6
scenario profile context	Profile evidence enters every decision	5	5/5	0/5
tool eligibility gate	Incompatible tools remain blocked	5	5/5	1/5

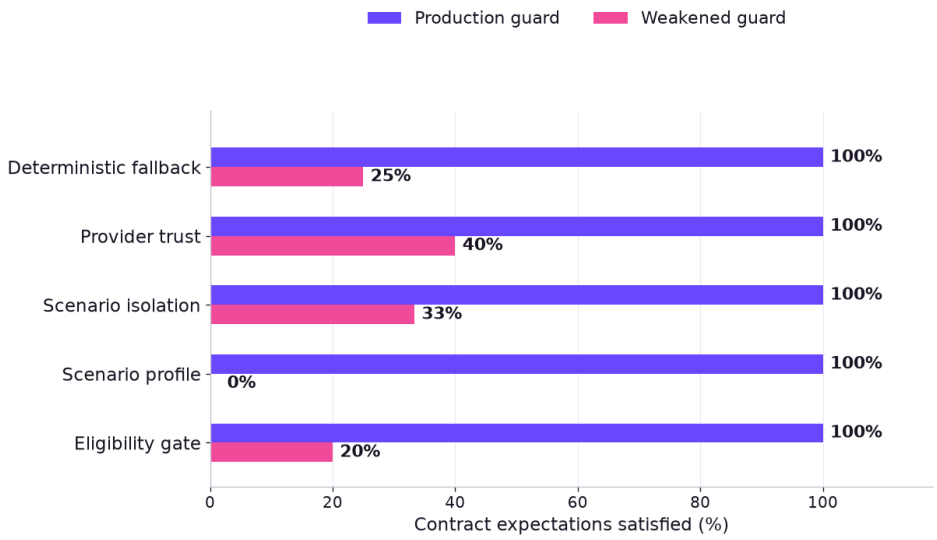

Figure 3. Constructed deterministic software-contract probes for each AURORA guard. The weakened comparison measures fail-closed boundary enforcement, not simulator performance or causal effect.

4.4 Fail-closed outcome equivalence

Five fixed seed blocks compare three matched-budget arms: direct portfolio, provider-error fallback, and invalid-response fallback. The campaign runs 15 Jobs and persists 579 Trials. Both fallback paths match the direct arm in all 10 blockwise comparisons across Candidates, Trials, budget, winner, holdout loss, failure count, and evidence completeness. All 15 arms reach 100% evidence completeness; a runtime socket guard observes zero network connection attempts and no real credential is used. By freezing the task, seed, design, budget, mock adapter, terminal accounting, and equivalence fields, the protocol isolates the recovery route as its only declared difference from the direct comparator.

Table 4: Fail-closed outcome equivalence

Arm	Job runs	Persisted Trials	Outcome vs direct
Direct deterministic portfolio	5	193	Reference arm
Provider-error fallback	5	193	5/5 exact
Invalid-response fallback	5	193	5/5 exact

The campaign establishes deterministic fallback equivalence on MockSimulatorAdapter under the frozen task, seeds, budget, adapter, and comparison fields. Equality covers persisted Candidates and Trials, selected winner, holdout loss, failure count, evidence completeness, and absence of network use; it does not imply that every future provider error is recoverable. Because all three arms execute the same deterministic portfolio after routing, the result does not show LLM superiority, causal AURORA benefit, PX4/Gazebo performance, controller quality, or flight safety. That bounded conclusion is replayable from retained Candidates, Trials, receipts, and comparison fields under the frozen protocol, using the archived JSON and CSV exports as the independent audit surface. A future recovery claim must rerun the matched design with an actual provider failure class and retain any arm-specific divergence instead of assuming that mock equality generalizes. This is the minimum evidence needed to distinguish recovery coverage from recovery performance.

4.5 Tool-use behavior

Eligibility is computed before routing. CMA-ES and the deterministic portfolio remain general fallbacks. Constrained multi-objective BO is admitted only for constrained or multi-objective tasks; multi-fidelity BO requires reducible replicates; TuRBO and surrogate CMA-ES require scored feasible history; and SAASBO or BIPOP CMA-ES require their respective high-dimensional or restart preconditions. A missing precondition removes the tool rather than asking the model to reason around it. The current 24-case distribution therefore describes qualified routing coverage under Evidence 2.7 and Prompt 1.6, not the relative optimization quality of those methods or a causal improvement over earlier prompt freezes [*4*, *6*, *7*, *8*]. Its frozen selections are portfolio 10, TuRBO 5, constrained MOBO 3, surrogate CMA-ES 2, BIPOP-CMA-ES 2, SAASBO 1, and multi-fidelity MOBO 1. These are route frequencies, not tool-quality estimates; every response is checked against the phase-qualified manifest before dispatch, and no method-specific superiority follows without physical outcomes. Figure 4 is therefore a coverage diagnostic for the validated router surface, not an empirical ranking or outcome comparison among the seven selected method families; it does not measure how any method changes controller behavior under a shared scenario, seed, and execution budget, and it cannot establish repeatability beyond the frozen routing corpus itself.

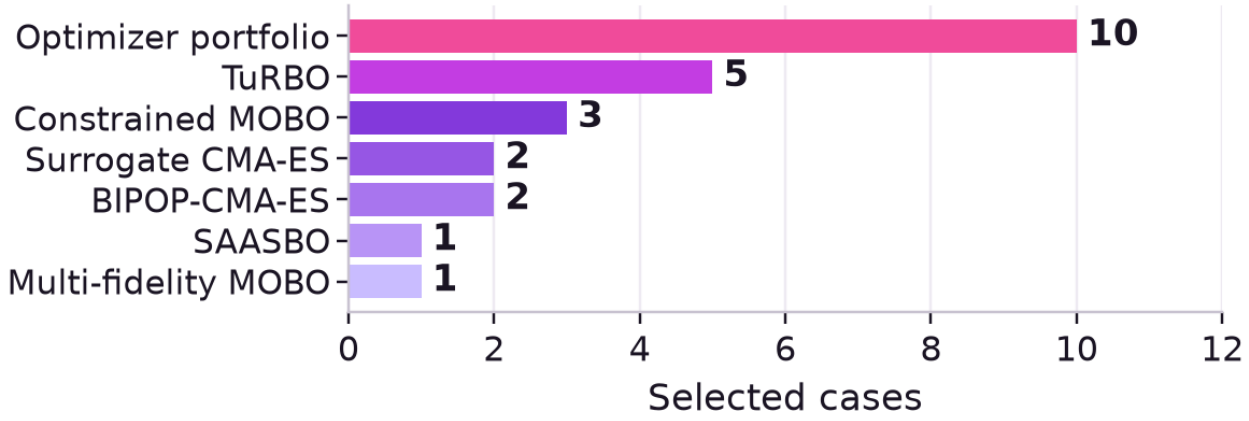

Figure 4. Tool distribution across the current 24-case Evidence 2.7 / Prompt 1.6 development freeze; counts describe selection frequency rather than optimizer quality or causal prompt improvement.

4.6 Synthetic integration campaign

This verified proxy experiment is deliberately labeled non-physical. It exercises compilation, candidate generation, deterministic evaluation, persistence, selection, holdout accounting, export, and evidence recomputation across ten scenario schemas. The local adapter uses frozen seeds and coefficients; no PX4, Gazebo, vehicle dynamics, network provider, or customer credential is involved. The result tests orchestration and evidence flow only; it does not establish controller quality, simulator fidelity, physical performance, deployment safety, or simulation-to-reality transfer.

Simulation-queue behavior is also an experimental-validity concern. Execution Claim Gate 1.1 binds each authorized Trial to a content-addressed snapshot, while Scheduling Gate 1.0 admits the least-served eligible Job's next FIFO Trial. Regressions cover concurrent claimants, physical-attempt accounting, retry ownership, and alternating claims across paired Jobs. These checks do not improve the proxy objective; they prevent reported budgets and cohort comparisons from being distorted by duplicate execution, starvation, or shared-Runtime queue monopolization. The ledger retains every authorized claim and terminal disposition for independent replay after every failure, retry, recovery, and ownership transition.

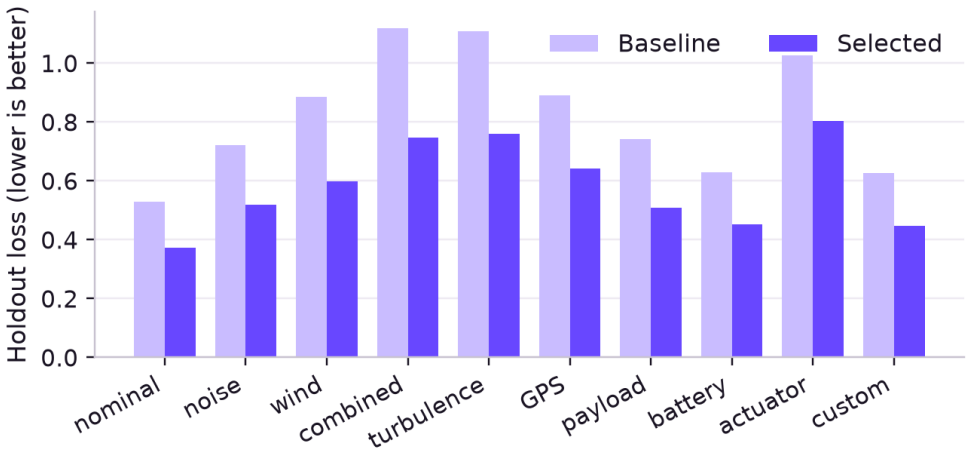

Figure 5. Baseline and selected holdout loss across ten frozen synthetic scenarios; evidence metadata records a mock simulator, false physical fidelity, and no provider network access.

Across the frozen 61-candidate mock campaign, aggregate holdout loss fell from 0.82811 to 0.58525 (29.327% relative). All ten named scenarios improved; scenario-wise relative reductions ranged from 21.8% to 33.2%. The selected candidate matched the exhaustive mock oracle in every scenario under the frozen proxy objective, candidate grid, and deterministic tie-break rule. The exporter recomputes the aggregate and per-scenario values from all persisted Trials and confirms that the selected Candidate belongs to the evaluated set. These measurements verify the closed synthetic campaign only; they do not predict the size or direction of improvement in a physical simulator under any flight condition.

The artifact declares physical_fidelity=false, simulator_backend=mock, and provider_network_access=false. Its scenario scores are generated by a frozen deterministic proxy objective rather than flight dynamics, sensors, actuator models, or environmental plugins. The claim is limited to orchestration, artifact completeness, and optimizer selection within that proxy. It is not PX4/Gazebo evidence, real-aircraft evidence, sim-to-real evidence, a safety claim, or a basis for physical deployment decisions; any later physical claim therefore requires independent, provenance-bound PX4/Gazebo measurements under the frozen release protocol with matched seeds and retained failures and independent recomputation externally.

4.7 Scenario-wise proxy behavior

Scenario-wise reporting exposes heterogeneity that a single aggregate loss would conceal. The heat strip and the quantitative table use the same ten frozen mock scenarios, selected Candidate, deterministic objective, and persisted holdout records as the integration campaign above. Values are paired within scenario: baseline and selected losses are recomputed from the ledger before the relative reduction is calculated. The resulting range shows that every declared proxy scenario improved under this closed campaign, while preserving the same non-physical boundary. These rows do not estimate uncertainty across vehicles or simulators and cannot be interpreted as PX4/Gazebo, real-aircraft, safety, or sim-to-real evidence.

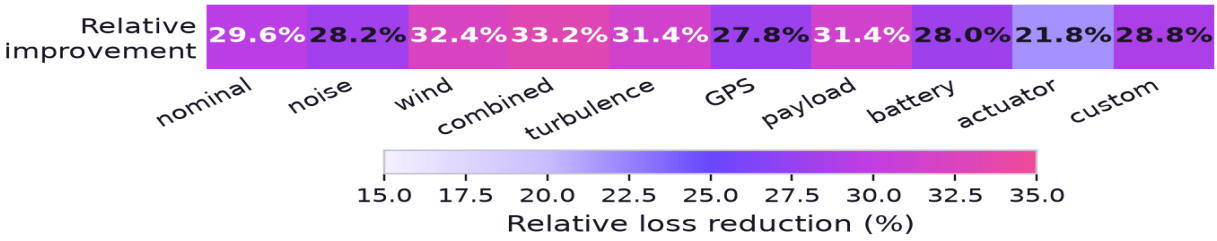

Figure 6. Relative loss reduction by mock scenario.

The strip should be read as a paired descriptive view of the frozen proxy campaign, not as a confidence interval or cross-vehicle generalization result. Each cell recomputes the relative change from the baseline and selected loss stored for the same named scenario, so a row cannot borrow evidence from a different seed, scenario, or Candidate. The narrow range is useful because it exposes whether one scenario dominates the aggregate, yet it remains a property of this deterministic mock objective. Publication-grade claims still require independent seeds, physical simulator receipts, retained failures, and uncertainty computed from the frozen PX4/Gazebo campaign rather than synthetic proxy assumptions.

Table 5: Scenario-wise synthetic holdout results

Scenario	Baseline	Selected	Reduction
nominal	0.5299	0.3733	29.55%
noise perturbed	0.7221	0.5185	28.20%
wind perturbed	0.8863	0.5992	32.39%
combined perturbed	1.1193	0.7474	33.23%
turbulence	1.1093	0.7609	31.41%
gps dropout	0.8899	0.6422	27.83%
payload changed	0.7429	0.5093	31.44%
battery degraded	0.6288	0.4530	27.96%
actuator delay	1.0262	0.8024	21.81%
custom	0.6264	0.4463	28.75%

4.8 Physical scenario evidence boundary

Constant horizontal wind for the bundled x500 family has been verified in a dedicated pinned WSL Runtime using PX4 commit 6ea3539157ca358c70a515878b77077af7d4611d and Gazebo 8.14.0. Acceptance requires trial-local source hashes, exact /wind_info read-back, and expanded runtime SDF evidence for the spawned instance. The signed installer Runtime has not yet passed this acceptance campaign, so the capability is not yet a released-customer claim. Static-obstacle injection has an implemented request and reply contract but awaits the same signed-Runtime acceptance. Gust and turbulence, GPS or sensor degradation, and battery, payload, or actuator-delay effects remain outside every customer-Runtime or physical-performance claim until complete simulator bindings, receipts, and acceptance evidence are frozen.

- Verified boundary — constant horizontal wind for x500 is PX4_GAZEBO evidence only inside the pinned dedicated WSL Runtime; it binds the source and effect checks recorded for that environment, while the signed installer Runtime remains unaccepted and outside every released customer-performance claim or distribution certificate in any channel.
- Implemented boundary — static-obstacle injection and reply validation exist in source, but implementation alone supplies neither executed scenario evidence nor a performance result; no released customer claim is made until the signed-Runtime campaign returns a complete accepted receipt with independently recomputed, failure-retaining outcomes for release.
- Pending boundary — gust, turbulence, GPS or sensor degradation, battery, payload, and actuator-delay effects remain excluded from physical claims until each disturbance has a complete simulator binding, trial-local source and effect evidence, an accepted receipt, and independently reviewed signed-Runtime results for that exact vehicle and scenario.
- Qualification receipt — pin the customer Runtime, PX4/Gazebo commits, vehicle/world assets, controller schema, disturbances, seeds, budgets, baseline arms, failure taxonomy, and independently recomputed outcomes for claim admission; retain every excluded Trial and acceptance decision in that versioned manifest with reason and artifact state.
- Denominator rule — retain every timeout and failed Trial; a missing receipt stays unavailable rather than being interpreted as a safe or successful run, and aggregate rates must preserve that disposition across every arm, scenario, seed block, and independently recomputed publication table without dropping unavailable runs from the denominator or seed.

5. Evaluation protocol for publication-grade results

The main benchmark will compare methods under the same pinned Runtime, controller catalog, starting design, train/holdout split, trial budget, failure thresholds, and blocked randomization schedule. Final-test manifests must be frozen before any arm observes their outcomes. The benchmark ledger must bind the Runtime image, simulator and controller commits, scenario files, parameter domains, seed blocks, initial design, stopping rule, failure-code mapping, and metric implementation. Each arm receives equal access to training outcomes and identical terminal accounting, while the final-test partition remains write-protected. Reported aggregates must be independently recomputed from persisted Trials with uncertainty, effect size, and failures retained in the denominator. The same ledger must retain every excluded run and the exact reason its result remained ineligible during external recomputation and final publication stage.

The controlled comparison should progress from a default deterministic policy to fixed CMA-ES, a specialist chosen before the final test, the deterministic portfolio, an AURORA arm with recovery memory disabled, and the complete AURORA system. Every arm must share the same scenario set, initialization, trial budget, and terminal accounting. Specialist selection must be frozen before final-test outcomes are visible, portfolio comparisons require a matched exploration budget, and the memory ablation must retain identical routing inputs. This sequence isolates increasingly specific system contributions without treating a method-name difference as causal evidence or attributing it to AURORA without evidence.

- Primary outcomes bind frozen holdout loss, feasible-campaign rate, baseline improvement, time-to-target, and terminal failure rate to every reported arm and seed block, with paired uncertainty and effect size computed from the retained scenario-level comparison units. These units govern preregistered acceptance and arm-to-arm inference at release.
- Secondary outcomes: wall time, model tokens and cost, router latency, recovery success, scenario-wise regret, and artifact completeness for every frozen comparison arm and seed block, reported alongside termination cause and evidence eligibility so efficiency cannot hide a higher failure burden or incomplete audit surface at publication time or review.
- Use at least five independent seeds for engineering diagnostics and preferably ten for report-level comparisons across every reported arm; freeze seed blocks before final execution and publish paired scenario-level uncertainty before making any superiority or repeatability claim across the preregistered final-test matrix and all baseline arms in scope.
- Keep timeouts, crashes, infeasible terminations, and failed Trials in the denominator under their predeclared terminal categories; never impute a successful loss, discard a difficult seed, or replace a missing receipt with an optimistic outcome, and never reclassify an unavailable execution after completed-arm results or rankings become visible during audit.
- Report paired uncertainty, effect sizes, sample counts, and failure dispositions for each arm and scenario family; never publish superiority from one seed, one track, a post-hoc metric, or an unregistered subset of completed runs.

6. Product reliability and operations

The desktop readiness gate is part of the product's technical contract. The progress bar may report completion only after every check that DroneDream can perform at launch has reached a terminal pass state. The entry action must not become visible while a required check is pending or failed. After entry, runtime failures still need explicit diagnostics because network loss, disk damage, permission changes, and external service changes cannot be permanently prevented. The gate therefore certifies a timestamped launch snapshot rather than promising an error-free future session, and post-entry faults must remain recoverable, attributable, and visible without silently invalidating prior work for every affected run. Its receipt should preserve probe timestamps, dependency versions, the service endpoint, account disposition, and the first failing stage. Recovery verification should replay install, upgrade, repair, offline, and damaged-cache paths without allowing stale success to satisfy a fresh launch or conceal the newly failing dependency under the same receipted probe sequence. The receipt must also retain the exact recovery action and the first post-recovery probe so a green launcher state remains attributable to one tested environment rather than becoming a generic assertion of platform health after repair or upgrade.

Readiness is a conjunction, not a progress animation. The launcher requires a fresh supported-system probe, one owned and compatible Runtime, a healthy local service, no mandatory update, and an account accepted by the local API. Missing, stale, mismatched, unreachable, or still-checking states remain blocking. Only the current gate may mark readiness complete, change the environment badge to CHECKED, and reveal the entry action. Quantitative release evidence still needs time-to-ready p50/p95, false-ready rate, and a clean Windows/upgrade matrix across fresh install, upgrade, repair, and duplicate-download paths before any release is accepted for public distribution on a supported Windows architecture.

Exit and account operations use the same recoverability principle. Closing warns about an active simulation or unsaved draft, stops work that cannot survive process termination, and preserves a recoverable draft whenever possible without trapping the user in the application. Managed-model usage consumes the included quota through a server-side ledger before BYOK is required; provider receipts must later be reconciled against that ledger. Remaining evidence therefore concerns draft-recovery and exit success rates plus quota-receipt consistency, not a new product capability claim, across every managed-provider transaction, recovery path, and audit replay retained by the service ledger.

7. Limitations and next experiments

AURORA's present evidence establishes enforceable software boundaries and reproducible mock-campaign behavior, but it does not yet establish broad optimization superiority or customer-Runtime physical-simulator performance. The distinctions below are active claim constraints, not generic future-work disclaimers: each one identifies the evidence unit that is missing, the confounder that remains uncontrolled, or the environment that has not been accepted. Keeping those boundaries explicit prevents development routing accuracy, deterministic guard probes, and synthetic proxy gains from being combined into a stronger conclusion than their protocols support. The next experiments are therefore ordered around frozen matched budgets, independent recomputation, signed Runtime acceptance, and scenario-level uncertainty over time.

- No result in this report establishes real-aircraft safety, certification, or direct simulation-to-reality transfer; those claims require separately governed hardware evidence, operational risk controls, and accepted flight testing outside the present software-contract scope and do not inherit assurance from the mock, WSL, or policy-only artifacts reported here.
- The current 24-case Evidence 2.7 / Prompt 1.6 freeze qualifies development routing only. The preserved Prompt 1.5 results of 19/24 and 21/24 expose stochastic spread; these unpaired freezes cannot support a causal prompt-lift, optimizer-quality, or simulator-quality claim without paired calls, fixed order, identical model snapshot, and case-level uncertainty.
- The current 16-case policy holdout verifies deterministic eligibility only; its published labels are not permanently blind and require a future concealed benchmark with sealed labels, preregistered scoring, independent custody, and a frozen separation between development examples and final evaluation cases under external final-test custody and audit.
- The source-contract ablation demonstrates named guard behavior, not causal optimization-quality lift; each weakened comparator changes a software protection under constructed probes and therefore cannot estimate controller performance, simulator fidelity, or benefit under real execution noise across matched scenario and seed blocks with uncertainty.
- The frozen four-arm mock ablation changes protocol behavior in 15/15 full-arm comparisons, but its 5/5 receipt-only-memory isolations remain inconclusive; the scripted policy does not authorize a causal performance claim outside a physical, matched-budget, multi-seed intervention with independent outcome recomputation and retained failures.
- The 10/10 fallback result verifies deterministic recovery on a frozen mock protocol, not that AURORA improves objective value beyond recovery equivalence or supports causal attribution; all arms converge on the same deterministic portfolio, so the retained comparison isolates dispatch recovery rather than optimization quality or causal lift.
- The ten-scenario campaign is synthetic mock evidence used to exercise orchestration and evidence flow; it is not physical scenario coverage, vehicle-dynamics validation, disturbance fidelity, or a substitute for signed-Runtime PX4/Gazebo outcomes. It cannot qualify vehicle dynamics, disturbance realism, or control quality in the signed Runtime.
- The pinned steady-wind result belongs to a dedicated WSL Runtime and is not yet accepted in the signed customer Runtime; release qualification still requires matching source, effect, scenario, and receipt checks in the distributed environment. Until replay passes, the WSL observation remains a prerequisite, not a release result for public use.
- Publication-grade optimizer comparisons, physical component ablations, multi-seed confidence intervals, cost/latency measurements, and UX telemetry remain pending before customer qualification, and each study must retain failures, matched budgets, frozen manifests, and independently recomputed outputs under the declared evidence class.
- External baselines such as expert manual tuning or PX4 AutoTune must be included only when controller, vehicle, track, budget, metrics, and failure accounting are genuinely aligned; unmatched access to resets, prior knowledge, simulator time, or terminal exclusions must be reported as a protocol difference rather than hidden inside a pooled ranking.

7.1 Threats to validity

The report's evidence units answer different questions and cannot be pooled into one score. Routing cases measure tool eligibility; source probes test whether named guards reject constructed violations; fallback arms measure recovery equivalence; and synthetic scenarios measure a deterministic proxy. Substituting one unit for another creates a construct-validity error even when all recorded values are correct. Because the same implementation sometimes generates, executes, and verifies an artifact, the next campaign also requires independent recomputation, immutable manifests, negative controls, and failure injection before an optimization-quality comparison is described as causal in publication.

Statistical validity requires the scenario, not an optimizer Trial, to remain the primary comparison unit. Trials in one trajectory share initialization, history, stopping conditions, and candidate ancestry, so treating them as independent understates uncertainty. A publication result must freeze seed blocks, report paired scenario-wise differences, retain terminal failures in the denominator, and present effect sizes with intervals rather than one aggregate percentage. Hyperparameters, method choice, and stopping rules must be fixed before the final partition opens; exploratory findings remain useful but cannot replace the frozen confirmatory comparison across every arm or its reported causal estimate.

Operational reproducibility requires more than code and a controller vector. A repeatable run must bind the owned Runtime image, PX4/Gazebo commits, vehicle and world assets, controller schema, scenario manifest, optimizer settings, seeds, environment, and evidence verifier. It must retain cancellations, timeouts, rejected callbacks, and post-launch changes rather than successful Trials alone. The ledger already supplies the source commit, test totals, digests, and reproduction command; the remaining gap is a signed customer-Runtime campaign executable on another machine without repository-specific state, hidden assumptions, operator interpretation, or independent auditability during replay.

External validity is bounded by controller, vehicle, world, and disturbance combinations actually executed. Many Trials in one x500 setup do not cover fixed-wing vehicles, payload changes, actuator saturation, estimator faults, or other controller structures. A broader benchmark therefore needs a declared scenario matrix with held-out combinations, not a convenient collection of worlds. Every cell must explain its distinct operating condition, mutable parameters, and invariant feasibility constraints. Results must be aggregated within and across vehicle families so a strong mean cannot conceal a systematic failure in one operationally important subgroup or declared scenario family under final evaluation.

Measurement validity also requires more than scalar holdout loss. The protocol must retain trajectory error, settling time, overshoot, control effort, saturation, constraint violations, termination cause, and wall-clock cost, with units, sampling rules, preprocessing versions, and direction of improvement. A controller can lower an aggregate score by accepting oscillation or excessive actuator demand, so feasibility remains a gate rather than a narrative caveat. Primary metrics and admissibility rules must be registered before optimization, while diagnostics remain visible for failure analysis; post-hoc metric choice cannot redefine a failed campaign as successful after execution once final outcomes become visible.

The next campaign prioritizes fixed-budget PX4/Gazebo outcomes, physical component-level AURORA ablations, failure injection, and the readiness/exit matrix. It freezes the Runtime, controller catalog, scenario manifests, seed blocks, budgets, eligibility policy, and terminal taxonomy before any final arm runs. Matched baselines include fixed CMA-ES, a preselected specialist, the deterministic portfolio, and isolated AURORA variants. Signed JSON and CSV artifacts will replace planned result sections without hiding failures or inflating evidence beyond its physical and statistical boundaries. The retained ledger must also show which planned comparisons never executed and why in the retained receipt. A causal provider-prompt study must also pair Prompt 1.5 and Prompt 1.6 calls on identical cases, model snapshot, call order, budget, and scoring rule; retain every raw response; and report paired category changes with uncertainty instead of selecting the best freeze. That design separates the current routing qualification from any future causal prompt-lift claim. The causal estimator must use paired case-level contrasts and retain category-specific disagreements, refusals, and parse failures in the denominator. The released audit artifact must preserve every paired case, call order, and response disposition for independent recomputation.

8. Reproducibility and source ledger

Figures 2–6 trace to evidence bundle v7, canonical SHA-256 prefix `a0432d092d5e751a`; Figure 1 is a schematic. The bundle binds software subject `742b124` to evidence head `1bf83cc`. Its backend receipt records 1,147/1,147 tests in 788.78 seconds on the exact software subject commit; an 81-check focused supplement passed in 151.96 seconds, with both logs bound by SHA-256. Ruff and mypy remain unreceipted, and Windows Rust was not run. The website receipt links `a64f995` to `943f3b6` and records 322/322 tests, typecheck, lint, build, nine deployment tests, and deterministic 117-file output across two builds. Together the reference manifest, claim audit, and companion receipt bind both dependency chains, every displayed value, the report source, PDF, audits, and reproduction commands to immutable Git bytes for independent review and regeneration.

Execution Claim Gate 1.1 and Outcome Contract 2.20 bind the Trial/Candidate/Job snapshot and authorized scenario matrix; drift fails closed before evidence admission. Concurrent claims retain one accepted physical attempt, while least-served per-Job scheduling prevents queue monopolization. Prompt Schema 2.3 assigns same-type training cases provider-safe aliases while sealing raw case IDs and holdout evidence. Evidence 2.7 and Prompt 1.6 add verified observational reflection; the current provider freeze passes 24/24, while Prompt 1.5 histories remain visible at 19/24 and 21/24. This is development-routing qualification, not causal prompt evidence. The deterministic eligibility artifact passes 16/16 policy-holdout cases without network or model calls, and the fallback campaign matches the direct arm in 10/10 frozen comparisons across 15 Jobs and 579 Trials. The audit retains source hashes, reproduction commands, qualification scope, and artifact provenance for independent regeneration, replay, and release review against the two frozen dependency chains and their signed receipts [*1*]. Release review additionally binds the receipt schema, artifact serialization, and verifier version, so a later rebuild cannot silently accept a different evidence contract. Independent replay must recompute declared counts and hashes from Git object bytes and reject missing objects before comparing presentation-layer bytes or customer-facing claims. This two-commit publication pattern prevents a receipt from authenticating bytes that did not exist at the source freeze. Reviewers can rebuild the source subject, compare its hashes, and inspect the later artifact commit without trusting the working tree that produced either object. Any mismatch remains a new report revision rather than an editorial substitution during independent review.

9. Conclusion

DroneDream 1.0 positions AURORA as an evidence-gated Harness for UAV controller optimization in simulation: model reasoning interprets intent and coordinates eligible methods, while deterministic contracts retain authority over inputs, execution, evidence admission, and completion. The frozen release demonstrates the exercised software boundaries through routing, holdout-policy, guard-ablation, fallback-equivalence, claim-receipt, and case-separated feedback checks; it does not convert those checks into a claim of real-aircraft safety or general simulator superiority. The next decisive step is a signed customer-Runtime PX4/Gazebo campaign with frozen scenario manifests, matched budgets, paired seeds, retained failures, and independent recomputation. That campaign can test optimization quality and operational reproducibility without weakening the provenance and leakage boundaries already enforced by AURORA in later studies.

The current release therefore changes what future experiments can safely assert before it changes the empirical flight result itself. Claim v3, Outcome Contract 2.20, input/outcome/scenario drift rejection, and cross-Job fair scheduling make concurrent attempt counts, parameter provenance, and scenario eligibility independently auditable. A decisive external campaign must still freeze PX4/Gazebo versions, manifests, seed blocks, budgets, metrics, and baseline arms before execution; retain every timeout and failed Trial; and recompute signed outcomes outside the application process. This separation keeps AURORA's demonstrated software assurances distinct from the optimization-quality claims that only the next physical simulator study can establish under the frozen evaluation protocol without ambiguity.

References

[1] DroneDream, "Release evidence ledger," ver. 1.0, evidence bundle v7, software subject `742b124`, Jul. 2026.

[2] PX4 Development Team, "Multicopter PID tuning guide (manual/basic)," PX4 User Guide, 2026. [Online]. [Accessed: Jul. 27, 2026]. Available: *PX4 Docs*

[3] PX4 Development Team, "Simulation," PX4 User Guide, 2026. [Online]. [Accessed: Jul. 27, 2026]. Available: *PX4 Docs*

[4] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," Evol. Comput., vol. 9, no. 2, pp. 159–195, 2001, doi: 10.1162/106365601750190398. *DOI*

[5] D. Golovin et al., "Google Vizier: A service for black-box optimization," in Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, 2017, pp. 1487–1495, doi: 10.1145/3097983.3098043. *DOI*

[6] M. Balandat et al., "BoTorch: A framework for efficient Monte-Carlo Bayesian optimization," in Adv. Neural Inf. Process. Syst., vol. 33, pp. 21524–21538, 2020. *arXiv*

[7] D. Eriksson et al., "Scalable global optimization via local Bayesian optimization," in Adv. Neural Inf. Process. Syst., vol. 32, 2019. *arXiv*

[8] D. Eriksson and M. Jankowiak, "High-dimensional Bayesian optimization with sparse axis-aligned subspaces," in Proc. 37th Conf. Uncertainty Artif. Intell., 2021, pp. 493–503. *arXiv*

[9] A. Brohan et al., "RT-1: Robotics Transformer for real-world control at scale," arXiv:2212.06817, 2022. *arXiv*

[10] A. Brohan et al., "RT-2: Vision-language-action models transfer Web knowledge to robotic control," in Proc. 7th Conf. Robot Learning, 2023. *arXiv*

[11] Open X-Embodiment Collaboration et al., "Open X-Embodiment: Robotic learning datasets and RT-X models," in Proc. IEEE Int. Conf. Robot. Autom., 2024. *arXiv*

[12] M. J. Kim et al., "OpenVLA: An open-source vision-language-action model," arXiv:2406.09246, 2024. *arXiv*

[13] K. Black et al., "π0: A vision-language-action flow model for general robot control," arXiv:2410.24164, 2024. *arXiv*

[14] Physical Intelligence et al., "π0.5: A vision-language-action model with open-world generalization," arXiv:2504.16054, 2025. *arXiv*

[15] C. Zhang et al., "JoyAI-RA 0.1: A foundation model for robotic autonomy," arXiv:2604.20100, 2026. *arXiv*

[16] J. Yang et al., "SWE-agent: Agent-computer interfaces enable automated software engineering," in Adv. Neural Inf. Process. Syst., vol. 37, 2024. *arXiv*

[17] A. Novikov et al., "AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms," arXiv:2506.13131, 2025. *arXiv*

[18] DeepSeek-AI, "DeepSeek-V3 technical report," arXiv:2412.19437, 2024. *arXiv*

[19] A. Yang et al., "Qwen2 technical report," arXiv:2407.10671, 2024. *arXiv*

[20] K. Bousmalis et al., "RoboCat: A self-improving generalist agent for robotic manipulation," Trans. Mach. Learn. Res., 2023. *arXiv*

[21] O. M. Team et al., "Octo: An open-source generalist robot policy," in Proc. Robotics: Science and Systems, 2024. *arXiv*

[22] H. Walke et al., "BridgeData V2: A dataset for robot learning at scale," in Proc. Conf. Robot Learning, 2023. *arXiv*

[23] A. Khazatsky et al., "DROID: A large-scale in-the-wild robot manipulation dataset," in Proc. Robotics: Science and Systems, 2024. *arXiv*

[24] A. Nasiriany et al., "RoboCasa: Large-scale simulation of everyday tasks for generalist robots," in Proc. Robotics: Science and Systems, 2024. *arXiv*

[25] Y. Chen et al., "RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation," arXiv:2506.18088, 2025. *arXiv*

[26] AgiBot-World Contributors et al., "AgiBot World Colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems," arXiv:2503.06669, 2025. *arXiv*

[27] C. Chi et al., "Diffusion Policy: Visuomotor policy learning via action diffusion," Int. J. Robot. Res., vol. 42, no. 13, pp. 1172–1198, 2023. *arXiv*

[28] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learn. Representations, 2023. *arXiv*

[29] T. Schick et al., “Toolformer: Language models can teach themselves to use tools,” in Adv. Neural Inf. Process. Syst., vol. 36, 2023. *arXiv*

[30] N. Shinn et al., “Reflexion: Language agents with verbal reinforcement learning,” in Adv. Neural Inf. Process. Syst., vol. 36, 2023. *arXiv*

[31] D. R. Jones, M. Schonlau, and W. J. Welch, “Efficient global optimization of expensive black-box functions,” J. Global Optim., vol. 13, no. 4, pp. 455–492, 1998, doi: 10.1023/A:1008306431147. *DOI*

[32] J. Snoek, H. Larochelle, and R. P. Adams, “Practical Bayesian optimization of machine learning algorithms,” in Adv. Neural Inf. Process. Syst., vol. 25, 2012. *arXiv*

[33] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, “Taking the human out of the loop: A review of Bayesian optimization,” Proc. IEEE, vol. 104, no. 1, pp. 148–175, Jan. 2016, doi: 10.1109/JPROC.2015.2494218. *arXiv*

Appendix A. Release-contract verification

The matrix separates receipted software checks from product claims. A.1 and A.2 define protected mutation and report-export contracts; Table 6 reports frozen receipts and the recomputed layout contract. Each count is source-bound and measures neither controller quality, physical-simulator fidelity, real-flight safety, nor causal optimization benefit. The map prevents software release verification from being mistaken for physical or optimizer performance; unsupported cells remain unavailable, never normalized into passing engineering evidence or any public customer qualification claim.

Table 6: Release-contract verification matrix

Verification surface	Cases	Passed	Claim boundary
Subject-focused supplement	81	81	Outcome/report/evidence paths
Frontend validation receipt	322	322	Tests plus typecheck/lint/build
Report layout contract	Recomputed	Required	Rendered body/list policy
Backend validation receipt	1,147	1,147	Exact-final full run; 81-check focused supplement

A.1 Experiment-report and reflection evidence policy

Downloadable tuning reports project retained evidence rather than infer narratives. They preserve generation ancestry, parameter deltas, metrics, terminal Trial counts, and proposal rationale; missing lineage remains unavailable. The authenticated server snapshot fixes export tier and blocks redirect-based entitlement bypass. Reflection is fail-closed: the execution receipt must match the exact modern Candidate cohort before AURORA exposes completion, accepted work, domain failures, feasibility, incumbent change, or observed improvement. Legacy, incomplete, or drifted cohorts keep the block unavailable; holdout values stay sealed, zero-dispatch stays not applicable, and observations never become causal tool credit.

A.2 Frozen report projection

The exporter rebuilds persisted evidence, not interface state. Its boundary binds the Job specification, ordered Candidate generations, parameter snapshots, objective and constraint metrics, terminal Trial taxonomy, proposal rationale, and server-resolved entitlement. It retains attempted iterations, admitted observations, parameter changes, and the holdout Candidate; missing receipts stay unavailable, failed Trials stay in the denominator, and prose cannot invent causes. The result exposes completed, failed, and unavailable observations rather than curating success. Free-account pages receive a native vector seal after pagination; Plus and Pro omit it only with authenticated entitlement. Redirects, stale state, or modified requests cannot alter that decision. The contract governs tier, watermark cardinality, iteration trace, missing evidence, and stable bytes, and does not qualify controller quality, simulator performance, or real-aircraft behavior in any evaluation.

A.3 Qualification handoff

A claim advances only when evidence class, prompt schema, source commit, Runtime identity, and acceptance command are frozen. Evidence 2.7 / Prompt 1.6 meets this boundary for the current 24/24 development freeze; the unpaired Prompt 1.5 histories (19/24 and 21/24) show stochastic variation, not causal lift. A causal study must pair calls, budgets, and seeds. Physical qualification separately requires matched budgets and seeds, retained failures, pinned PX4/Gazebo commits, and independently recomputed outcomes in the signed Runtime. Until then, AURORA’s claim is limited to software-contract behavior, not optimizer superiority, physical-simulator fidelity, or flight performance.

A.4 Frozen component outcome ablation

The preregistered offline ablation locks five seed blocks across full AURORA, no decision memory, no observed-outcome reflection, and a fixed portfolio. Its 20 Jobs persist 554 Trials with complete evidence, zero network calls, and no real credential. Across the 15 paired arm comparisons, five expose an observed protocol difference and ten show no observed protocol difference under the frozen local router. The five receipt-only-memory isolations remain explicitly inconclusive rather than being converted into a causal component claim under this deliberately bounded offline decision protocol.

Table 7: Frozen four-arm component outcome ablation

Arm	Jobs	Trials	Tool sequence	Finding
Full AURORA	5	120	constrained MOBO to TuRBO	Reference
No decision memory	5	120	constrained MOBO to portfolio	No observed difference
No outcome reflection	5	120	constrained MOBO to portfolio	No observed difference
Fixed portfolio	5	194	portfolio to portfolio	Observed difference
Receipt-only isolation	5 pairs	—	matched route and outcome	Inconclusive

A.5 Phase-qualified execution surface

Capability eligibility answers whether a tool can honor the declared Job shape; the phase policy independently answers whether that tool should be executable now. Evidence 2.7 intersects the capability-qualified set with the receding-plan phase before schema generation, provider manifest construction, response validation, trace verification, or dispatch. Prompt 1.6 and Decision Trace 1.3 therefore persist the exact surface shown to the model. Every phase retains the deterministic portfolio fallback, and a response naming any filtered tool is rejected rather than repaired into an executable decision. The retained trace therefore explains every admitted and excluded tool during later replay and independent review.

Fidelity remains a separate execution contract. Multi-fidelity methods may screen Candidates below full fidelity during exploration or diversification, but a verification decision forces every reduced-fidelity-capable strategy to fidelity 1.0. The final generation and last available capacity also preserve a full-fidelity path, while the ordinary acceptance, claim-receipt, scenario, and holdout gates remain unchanged. This prevents an inexpensive screening outcome from becoming the final winner merely because its optimizer was otherwise eligible for the current Job and scenario profile. The receipt must therefore record both screening fidelity and the full-fidelity verification used for terminal selection.

Table 8: Phase-qualified search-role surface

Plan phase	Role count	Primary search intent	Fidelity rule
Exploration	6	Global, sparse, and multi-fidelity discovery	Budget policy
Recovery	4	Restart and deterministic recovery	Budget policy
Refinement	5	Local and surrogate exploitation	Budget policy
Diversification	7	Broad escape from stalled regions	Budget policy
Verification	5	Confirm the strongest eligible Candidate	**Fidelity 1.0**
Balanced	8	Preserve the complete qualified registry	Budget policy

A.6 Evidence-source ledger

A machine-checked ledger binds every displayed count, rate, scenario row, contract version, phase count, and Runtime boundary to immutable Git bytes and fails the build on drift. Router rows authorize eligibility claims only; mock campaigns authorize bounded software and synthetic-response claims; component ablations authorize observed differences under their frozen offline router. Backend and frontend receipts validate software, not provider, PX4/Gazebo, or flight behavior. Each reproducible value remains paired with the stronger unauthorized conclusion and the future experiment needed to support it. At release, the ledger also binds the generated PDF, layout audit, claim audit, and validation receipt to a source commit without self-reference. A verifier reads those bytes from Git objects, rechecks serialization and page-level gates, and fails if any dependency SHA, figure, assertion, or receipt field drifts. This keeps the website handoff limited to the final PDF and its hash while preserving source, scripts, manifests, and receipts under report ownership. That separation lets the website cite the artifact without acquiring authority to regenerate, alter, or reinterpret the report's evidence chain; any revised PDF requires a new report source commit, complete build, visual review, and validation receipt. The ledger also separates regeneration from interpretation: reproducing the same artifact does not authorize a reader to widen the claim boundary attached to any row. A changed source, dependency, or conclusion must receive a new subject commit, regenerated figures, and a fresh receipt instead of inheriting the prior report's validation status. This lifecycle keeps editorial revision, evidence revision, and product release as separately reviewable events under the same auditable release boundary.

Table 9: Evidence-source and claim-boundary ledger

Evidence surface	Scale	Source ID	Authorized claim
Provider routing freezes	19/24, 21/24, 24/24	Prompt 1.5 / 1.6	Current development routing only
Policy holdout	16/16	Policy holdout v3	Eligibility-set determinism
Source-contract ablation	25/25 vs. 6/25	Contract ablation v2	Fail-closed guard behavior
Fallback outcome campaign	15 Jobs, 579 Trials	Fallback campaign v1	Matched offline recovery
Component outcome ablation	20 Jobs, 554 Trials	Component ablation v2	Observed protocol differences
Synthetic scenario campaigns	2 × 61 Candidates	Coverage v3 / gen. v1	Seed and mixed-shift reporting